

Logistic Regression

Sandro Cumani

sandro.cumani@polito.it

Politecnico di Torino

Discriminative Linear Models – Logistic Regression

Logistic regression, despite its name, is a discriminative approach for classification

Rather than modeling the distribution of observed samples $X|C$, we directly model the class posterior distribution $C|X$

We need to define a model for the class posterior distribution $P(C = c|X = \mathbf{x})$

Discriminative Linear Models – Logistic Regression

For a 2-class problem, we have seen that the Gaussian model with tied covariances provides log-likelihood ratios that are linear functions of our data

$$l(\mathbf{x}) = \log \frac{f_{X|C}(\mathbf{x}|h_1)}{f_{X|C}(\mathbf{x}|h_0)} = \mathbf{w}^T \mathbf{x} + c$$

and the class log-posterior probability ratio is

$$\log \frac{P(C = h_1|\mathbf{x})}{P(C = h_0|\mathbf{x})} = \log \frac{f_{X|C}(\mathbf{x}|h_1)}{f_{X|C}(\mathbf{x}|h_0)} + \log \frac{\pi}{1 - \pi} = \mathbf{w}^T \mathbf{x} + b$$

The prior information has been absorbed in the bias term b .

Logistic Regression

Given $\mathbf{w}$ and b , we can compute the expression for the posterior class probability as

$$\begin{aligned}P(C = h_1|\mathbf{x}, \mathbf{w}, b) &= e^{(\mathbf{w}^T\mathbf{x}+b)}P(C = h_0|\mathbf{x}, \mathbf{w}, b) \\&= e^{(\mathbf{w}^T\mathbf{x}+b)}(1 - P(C = h_1|\mathbf{x}, \mathbf{w}, b))\end{aligned}$$

Solving for $P(C = h_1|\mathbf{x}, \mathbf{w}, b)$ we obtain

$$P(C = h_1|\mathbf{x}, \mathbf{w}, b) = \frac{e^{(\mathbf{w}^T\mathbf{x}+b)}}{1 + e^{\mathbf{w}^T\mathbf{x}+b}} = \frac{1}{1 + e^{-(\mathbf{w}^T\mathbf{x}+b)}} = \sigma(\mathbf{w}^T\mathbf{x} + b)$$

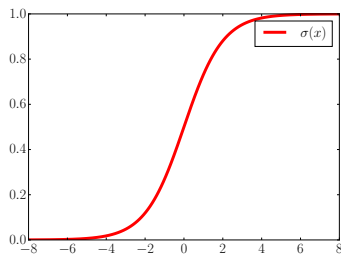
where

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

is called *sigmoid* function (or *logistic* function)

Logistic Regression

Sigmoid function:



Some properties of $\sigma(x)$ that will come useful later

- $1 - \sigma(x) = \sigma(-x)$
- $\frac{d\sigma(x)}{dx} = \sigma(x) (1 - \sigma(x))$

Logistic Regression

The equation $P(C = h_1 | \mathbf{x}, \mathbf{w}, b) = \sigma(\mathbf{w}^T \mathbf{x} + b)$ provides a model that allows computing the posterior probabilities for h_1 and h_0

The model assumes that decision rules are linear surfaces (hyperplanes) orthogonal to $\mathbf{w}$. The model parameters are $(\mathbf{w}, b)$

If we knew $(\mathbf{w}, b)$ then we could compute the predictive distribution for the class labels $P(C = h_1 | \mathbf{x}, \mathbf{w}, b)$

We have seen how we can compute estimates for $(\mathbf{w}, b)$ from a generative Gaussian model

However, in the following, we ignore the generative model for X and concentrate on the form of the class posterior probabilities

Again, we will follow a frequentist approach, i.e. compute an estimate for $\mathbf{w}$ and b from a set of training samples

Logistic Regression

We assume we have a labeled dataset

$$\mathcal{D} = [(\mathbf{x}_1, c_1), \dots (\mathbf{x}_n, c_n)]$$

We also assume classes are independently distributed as

$$C_i | \mathbf{x}_i, \mathbf{w}, b \sim C | \mathbf{x}_i, \mathbf{w}, b$$

The class-posterior model allows expressing the likelihood for the observed labels as

$$P(C_1 = c_1, \dots C_n = c_n | \mathbf{x}_1, \dots \mathbf{x}_n, \mathbf{w}, b) = \prod_{i=1}^n P(C_i = c_i | \mathbf{x}_i, \mathbf{w}, b)$$

We can apply a ML approach to estimate the model parameters that best describe the observed labels $(c_1 \dots c_n)$

Logistic Regression

Rather than estimating $\mathbf{w}$ from class-conditional likelihoods, we estimate the value of $\mathbf{w}$ that maximizes the likelihood of our training labels.

We assume that the label for class h_1 is 1, and the label for class h_0 is 0

Let

$$y_i = P(C_i = 1 | \mathbf{x}_i, \mathbf{w}, b) = \sigma(\mathbf{w}^T \mathbf{x}_i + b)$$

It follows that

$$P(C_i = 0 | \mathbf{x}_i, \mathbf{w}, b) = 1 - y_i = 1 - \sigma(\mathbf{w}^T \mathbf{x}_i + b) = \sigma(-\mathbf{w}^T \mathbf{x}_i - b)$$

Logistic Regression

The distribution for $C_i|\mathbf{x}_i, \mathbf{w}, b$ is a Bernoulli distribution

$$C_i|\mathbf{x}_i, \mathbf{w}, b \sim \text{Ber}(\sigma(\mathbf{w}^T \mathbf{x}_i + b)) = \text{Ber}(y_i)$$

The likelihood for our label set is

$$\begin{aligned}\mathcal{L}(\mathbf{w}, b) &= P(C_1 = c_1, \dots, C_n = c_n | \mathbf{x}_1, \dots, \mathbf{x}_n, \mathbf{w}, b) \\ &= \prod_{i=1}^n P(C_i = c_i | \mathbf{x}_i, \mathbf{w}, b) \\ &= \prod_i y_i^{c_i} (1 - y_i)^{(1-c_i)}\end{aligned}$$

Again, it's more practical to work with the log-likelihood

$$\ell(\mathbf{w}, b) = \log \mathcal{L}(\mathbf{w}, b) = \sum_{i=1}^n [c_i \log y_i + (1 - c_i) \log(1 - y_i)]$$

Logistic Regression

Our goal is the maximization of ℓ . We thus seek $\mathbf{w}^*, b^*$ that maximize $\ell(\mathbf{w}, b)$:

$$\mathbf{w}^*, b^* = \arg \max_{\mathbf{w}, b} \ell(\mathbf{w}, b) = \arg \max_{\mathbf{w}, b} \sum_{i=1}^n [c_i \log y_i + (1 - c_i) \log(1 - y_i)]$$

The ML solution is also the solution that minimizes the average *cross-entropy* between the distribution of observed and predicted labels

Rather than maximizing $\ell(\mathbf{w}, b)$, we can *minimize*

$$J(\mathbf{w}, b) = -\ell(\mathbf{w}, b) = \sum_{i=1}^n -[c_i \log y_i + (1 - c_i) \log(1 - y_i)]$$

Logistic Regression

The expression

$$H(c_i, y_i) = -[c_i \log y_i + (1 - c_i) \log (1 - y_i)]$$

represents the binary *cross-entropy* between the distribution of observed and predicted labels for the i -th sample

More in general, let P and Q be two distributions over the same domain

The cross-entropy between the two distributions is defined as

$$H(P, Q) = -\mathbb{E}_{P(x)} [\log Q(x)]$$

For discrete distributions, this can be expressed as

$$H(P, Q) = -\sum_{x \in \mathcal{S}} P(x) \log Q(x)$$

Logistic Regression

In our case, P is the empirical distribution of class labels, *from the point of view of an observer $\mathcal{E}$ who knows the actual label*:

$$P(C_i = 1 | \mathbf{X}_i = \mathbf{x}_i, \mathcal{E}) = \begin{cases} 1 & \text{if } c_i = 1 \\ 0 & \text{if } c_i = 0 \end{cases}$$

$$P(C_i = 0 | \mathbf{X}_i = \mathbf{x}_i, \mathcal{E}) = \begin{cases} 0 & \text{if } c_i = 1 \\ 1 & \text{if } c_i = 0 \end{cases}$$

or, equivalently

$$P(C_i = 1 | \mathbf{X}_i = \mathbf{x}_i, \mathcal{E}) = c_i, \quad P(C_i = 0 | \mathbf{X}_i = \mathbf{x}_i, \mathcal{E}) = 1 - c_i$$

i.e., a Bernoulli distribution with parameter c_i

Logistic Regression

Distribution Q is the distribution for the *predicted labels according to our recognizer $\mathcal{R}$*

$$Q(c) = P(C_i = c | X_i = \mathbf{x}_i, \mathcal{R}(\mathbf{w}, b))$$

i.e.

$$Q(1) = P(C_i = 1 | X_i = \mathbf{x}_i, \mathcal{R}(\mathbf{w}, b)) = y_i = \sigma(\mathbf{w}^T \mathbf{x}_i + b)$$

$$Q(0) = P(C_i = 0 | X_i = \mathbf{x}_i, \mathcal{R}(\mathbf{w}, b)) = 1 - y_i = 1 - \sigma(\mathbf{w}^T \mathbf{x}_i + b)$$

Logistic regression looks for the *minimizer of the average cross-entropy* between the distributions for the training set labels of an evaluator $\mathcal{E}$ *who knows the real label* and the distributions for the training set labels as *predicted by the model $\mathcal{R}(\mathbf{w}, b)$* itself

The cross-entropy is a measure of goodness of the predictions, and the evaluation is performed over the training data itself

Logistic Regression

The cross-entropy, as a function of Q , is minimized when $Q = P$

The cross-entropy can also be interpreted as a measure of the difference between P and Q

In our case, it measures how different is the predicted distribution $\text{Ber}(y_i)$ from the empirical label distribution $\text{Ber}(c_i)$ (the distribution of the evaluator $\mathcal{E}$)

Minimization of the average cross-entropy means we are looking for a label distribution that is (on average) as similar as possible to the empirical one

Alternatively, as we have seen, we can regard the process as maximization of the likelihood for the observed labels

Logistic Regression

Another interesting interpretation of the logistic regression objective can be obtained by rewriting the cross-entropy in terms of $z_i = 2c_i - 1$, i.e.

The terms z_i still represent class labels, however for samples of class h_1 we have $z_i = 1$, whereas for samples of class h_0 we have $z_i = -1$:

$$z_i = \begin{cases} 1 & \text{if } c_i = 1 \\ -1 & \text{if } c_i = 0 \end{cases}$$

Logistic Regression

The objective function that we want to minimize corresponds to the sum of n terms

$$J(\mathbf{w}, b) = \sum_i H(c_i, y_i)$$

where

$$H(c_i, y_i) = -[c_i \log y_i + (1 - c_i) \log(1 - y_i)]$$

Note that $H(c_i, y_i)$ is a function of c_i , but also of $\mathbf{w}$, b and $\mathbf{x}_i$, since

$$y_i = \sigma(\mathbf{w}^T \mathbf{x}_i + b)$$

Logistic Regression

Let $s_i = \mathbf{w}^T \mathbf{x}_i + b$. In terms of z_i we can rewrite H as

$$\begin{aligned} H(c_i, y_i) &= -[c_i \log y_i + (1 - c_i) \log(1 - y_i)] \\ &= \begin{cases} -\log \sigma(s_i) & \text{if } c_i = 1 \ (z_i = 1) \\ -\log(1 - \sigma(s_i)) = -\log \sigma(-s_i) & \text{if } c_i = 0 \ (z_i = -1) \end{cases} \\ &= -\log \sigma(z_i s_i) \\ &= -\log \sigma(z_i(\mathbf{w}^T \mathbf{x}_i + b)) \\ &= \log \left(1 + e^{-z_i(\mathbf{w}^T \mathbf{x}_i + b)} \right) \end{aligned}$$

Logistic Regression

The objective function can thus be rewritten as

$$\begin{aligned} J(\mathbf{w}, b) &= \sum_{i=1}^n H(c_i, y_i) \\ &= \sum_{i=1}^n \log \left(1 + e^{-z_i(\mathbf{w}^T \mathbf{x}_i + b)} \right) \\ &= \sum_{i=1}^n l(z_i(\mathbf{w}^T \mathbf{x}_i + b)) \end{aligned}$$

where

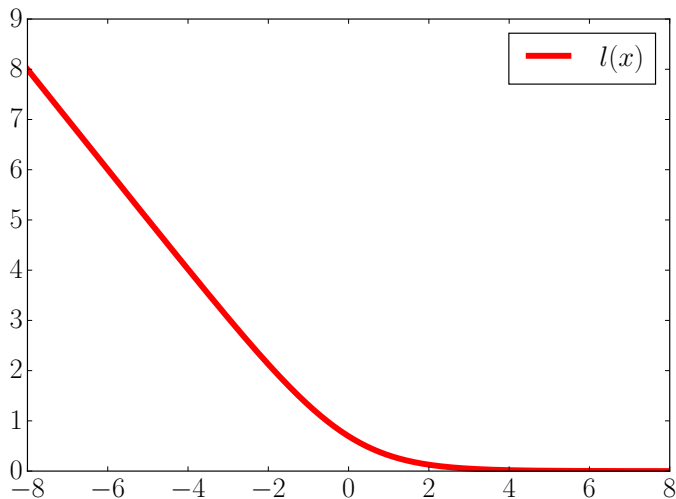
$$l(x) = \log(1 + e^{-x})$$

is the *logistic loss* function.

Our goal is to find the *minimizer* of $J(\mathbf{w}, b)$

Logistic Regression

Logistic loss



Logistic Regression

We can interpret the function as the *cost* of the prediction made with model $(\mathbf{w}, b)$ for each sample

Remember that class log-posterior probability ratio is

$$\log \frac{P(h_1|\mathbf{x}_i)}{P(h_0|\mathbf{x}_i)} = \mathbf{w}^T \mathbf{x}_i + b = s_i$$

Decision rules take the form $s_i \lessgtr t$

Since $s_i = \mathbf{w}^T \mathbf{x}_i + b$, decision rules are linear hyperplane orthogonal to the vector $\mathbf{w}$

s_i is related to the distance of the sample $\mathbf{x}_i$ from the separating surface

Logistic Regression

When s_i is positive, our classifier is favoring class h_1 , whereas negative s_i means we are classifying the sample as belonging to class h_0 .

The cost we pay for each sample is $l(z_i s_i)$

- The prediction and the actual class agree: $z_i = 1, s_i > 0$ or $z_i = -1, s_i < 0$. Then $z_i s_i > 0$, and we pay a low cost. The cost becomes exponentially smaller (asymptotically) as the absolute value of s_i increases (we move away from the separation surface)
- The prediction and the actual class disagree: $z_i = 1, s_i < 0$ or $z_i = -1, s_i > 0$. Then $z_i s_i < 0$, and we pay a cost that increases (asymptotically) linearly with s_i

Logistic Regression

We can thus interpret the logistic regression objective as a measure of an *empirical risk*¹. Our goal is minimizing the empirical risk

More in general, empirical risk minimization is a framework for the estimation of classification models which aims at minimizing an *empirical risk function* over our training data

Generalized risk minimization problem: minimize the risk $R(\theta)$

$$R(\theta) = \sum_i l(\mathbf{w}, \mathbf{x}_i, z_i)$$

where l is called loss (or cost) function, and θ are the parameters of the classification model, e.g. $\theta = (\mathbf{w}, b)$ in our case.

¹Empirical because it's computed on the observed samples

Logistic Regression

Logistic regression solutions cannot be computed in closed form

We will resort to numerical solvers

A numerical solver iteratively looks for the minimizer of a function

We will use the L-BFGS algorithm

The algorithm requires a function that computes the loss and its gradient with respect to w and b

In the laboratory we will see how to implement the minimization

Logistic Regression

If classes are linearly separable, the logistic regression solution is not defined

Linearly separable classes: there exist $\mathbf{w}$ and b such that all training samples lie on the correct side of the corresponding separation surface ($z_i > 0 \iff s_i > 0$)

In this case, we can make the values of s_i arbitrarily high by simply increasing the norm of $\mathbf{w}$ (and changing accordingly the value of b)

As we increase $\|\mathbf{w}\|$, the loss becomes lower, thus we are decreasing the objective function

The function does not have a minimum, but has an infimum $\inf J(\mathbf{w}, b) = 0$, corresponding to $\|\mathbf{w}\| \rightarrow \infty$

Logistic Regression

To make the problem solvable again, we can look for solutions with small norm by introducing a norm penalty to the objective function

The penalty is called *regularization* term

The objective function that we minimize is

$$\tilde{R}(\mathbf{w}, b) = \frac{\tilde{\lambda}}{2} \|\mathbf{w}\|^2 + \sum_{i=1}^n \log \left(1 + e^{-z_i(\mathbf{w}^T \mathbf{x}_i + b)} \right)$$

where $\tilde{\lambda}$ is a hyper-parameter that allows specifying the relative weight of the regularization term

Alternatively, we look for the minimizer of

$$R(\mathbf{w}, b) = \frac{\lambda}{2} \|\mathbf{w}\|^2 + \frac{1}{n} \sum_{i=1}^n \log \left(1 + e^{-z_i(\mathbf{w}^T \mathbf{x}_i + b)} \right)$$

where the risk is *averaged* over all samples, and λ is the regularization coefficient.

Logistic Regression

λ is a hyper-parameter, and should be selected as to optimize the performance of the classifier

Note that λ cannot be computed by minimizing R with respect to λ , as we would obtain the trivial solution $\lambda = 0$

The selection of good values for λ should thus be based on other approaches, such as *cross-validation*

The model is called *regularized* Logistic Regression, and is an example of a *regularized risk minimization* problem

$$R(\mathbf{w}, b) = \Omega(\mathbf{w}, b) + \frac{1}{n} \sum_{i=1}^n l(\mathbf{x}_i, z_i, \mathbf{w}, b)$$

Logistic Regression

The regularization term Ω (in our case $\frac{\lambda}{2} \|\mathbf{w}\|^2$) can be interpreted as a term that favors simpler solutions (we will see explicitly why small norm of $\mathbf{w}$ can be interpreted as a simpler solution when discussing Support Vector Machines)

Regularization allows reducing the risk of over-fitting the training data

Of course, if λ is too large, we will obtain a solution that has small norm, but is not able to well separate the classes

On the other hand, if λ is too small, we will get a solution that has good separation on the training set, but may have poor classification accuracy for unseen data (i.e. poor *generalization*)

Logistic Regression

Some considerations:

- The non-regularized model is invariant to linear transformations of the feature vectors.
- The regularized version of the model, on the other hand, is not invariant.
- It is therefore useful, in some cases, to pre-process data so that dynamic ranges of different features are similar
- Cross-validation can help in identifying good pre-processing strategies

Logistic Regression

Common *preprocessing strategies* that may be worth trying:

- Center the data (either using the training set, or a weighted mean — e.g. the average of class means): $\mathbf{x}'_i = \mathbf{x}_i - \boldsymbol{\mu}$
- Standardize variances (e.g. divide each feature by its own standard deviation computed over the training set): $\mathbf{x}'_{i,[j]} = \mathbf{x}_{i,[j]} / \sigma_{[j]}$
- Whiten the covariance matrix (i.e. normalize variances while making features uncorrelated): $\mathbf{x}'_i = \mathbf{A}\mathbf{x}_i$, where $\mathbf{A} = \boldsymbol{\Sigma}^{-\frac{1}{2}}$ and $\boldsymbol{\Sigma}$ is the training set covariance (a variation consists in replacing $\boldsymbol{\Sigma}$ with the within-class covariance)
- L2 (or length) normalization: $\mathbf{x}'_i = \frac{\mathbf{x}_i}{\|\mathbf{x}_i\|}$ (often after centering and whitening)

Logistic Regression

Some considerations:

- The Logistic Regression score can be interpreted as the logarithm of the ratio between class posterior probabilities
- It reflects the empirical class prior of the training data
- We can simulate different empirical priors π_T using a prior-weighted version of the model

$$R(w) = \frac{\lambda}{2} \|w\|^2 + \frac{\pi_T}{n_T} \sum_{i|z_i=1} l(z_i s_i) + \frac{1 - \pi_T}{n_F} \sum_{i|z_i=-1} l(z_i s_i)$$

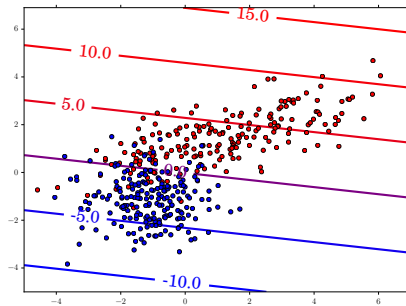
Logistic Regression

Some considerations:

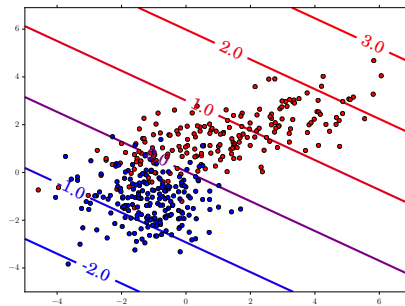
- If our application is characterized by the same effective prior π_T , the optimal decision should correspond to $\mathbf{w}^T \mathbf{x} + b \lesseqgtr 0$
- In theory, we can also recover log-likelihood ratios by subtracting from the score s the empirical prior log-odds of the training set $\log \frac{n_T}{n_F}$, where n_T is the number of samples of class $h_1(c_i = 1, z_i = +1)$ and n_F is the number of samples of class $h_0(c_i = 0, z_i = -1)$
- In some cases, especially if the dimensionality of the space is large, the score s may not provide the correct probabilistic interpretation, and optimal decisions may require either recalibration of the scores or selection of an optimal threshold based on a validation set

Logistic Regression

$\lambda = 0$



$\lambda = 1$



Logistic Regression

We test the model on MNIST digit pairs (e.g. 0 vs 1, 0 vs 2, ...)

MNIST — Average Pairwise EER for Logistic Regression

DimRed	$\lambda = 0$	$\lambda = 0.00001$	$\lambda = 0.001$	$\lambda = 0.1$	Tied Gau
RAW [768]	1.7%	1.4%	1.2%	2.0%	—
PCA [50]	1.4%	1.4%	1.4%	2.1%	1.7%
PCA [100]	1.3%	1.2%	1.2%	2.0%	1.5%

LogReg obtains better performance than the Gaussian model

Regularization is important, especially when we do not reduce the dimensionality (we have more parameters to estimate, so over-fitting is more severe)

If we regularize too much the model performs poorly again

Multiclass Logistic Regression

We now consider a problem with K classes, labeled from 1 to K

To extend the Logistic Regression model to multiclass tasks we start again from the form of the posterior likelihood ratios of the Linear Gaussian classifier with uniform priors

$$\log \frac{P(C = j|\mathbf{x})}{P(C = r|\mathbf{x})} = (\mathbf{w}_j - \mathbf{w}_r)^T \mathbf{x} + (b_j - b_r)$$

corresponding to pair-wise linear classification surfaces between class j and a reference class r

Notice that we over-parametrized the model by introducing an extra set of parameters $(\mathbf{w}_r, b_r)$, so that we do not have to explicitly enforce $\log \frac{P(C=r|\mathbf{x})}{P(C=r|\mathbf{x})} = 0$

Multiclass Logistic Regression

It follows that, for all classes $j \in 1, \dots, K$,

$$P(C = j|\mathbf{x}) = P(C = r|\mathbf{x})e^{(\mathbf{w}_j - \mathbf{w}_r)^T \mathbf{x} + (b_j - b_r)}$$

Remembering that

$$\sum_{j=1}^K P(C = j|\mathbf{x}) = 1$$

We have

$$P(C = r|\mathbf{x}) = 1 - \sum_{j \neq r} P(C = j|\mathbf{x}) = 1 - \sum_{j \neq r} P(C = r|\mathbf{x})e^{(\mathbf{w}_j - \mathbf{w}_r)^T \mathbf{x} + (b_j - b_r)}$$

Multiclass Logistic Regression

Therefore

$$\begin{aligned}P(C = r|\mathbf{x}) &= \frac{1}{1 + \sum_{j \neq r} e^{(\mathbf{w}_j - \mathbf{w}_r)^T \mathbf{x} + (b_j - b_r)}} \\&= \frac{1}{\sum_j e^{(\mathbf{w}_j - \mathbf{w}_r)^T \mathbf{x} + (b_j - b_r)}} \\&= \frac{e^{\mathbf{w}_r^T \mathbf{x} + b_r}}{\sum_j e^{\mathbf{w}_j^T \mathbf{x} + b_j}}\end{aligned}$$

Note that the model is completely specified (and actually over-parametrized) once we know the K vectors $\mathbf{w}_k$ and the terms b_k . Repeating the operations for all classes we obtain

$$P(C = k|\mathbf{x}) = \frac{e^{\mathbf{w}_k^T \mathbf{x} + b_k}}{\sum_j e^{\mathbf{w}_j^T \mathbf{x} + b_j}}$$

Function $f_i(\mathbf{s}) = \frac{e^{s_i}}{\sum_j e^{s_j}}$ is called *softmax*

Multiclass Logistic Regression

Given the model parameters

$$\mathbf{W} = [\mathbf{w}_1 \quad \dots \quad \mathbf{w}_K] , \quad \mathbf{b} = \begin{bmatrix} b_1 \\ \vdots \\ b_K \end{bmatrix}$$

the logistic regression model allows computing the probability of each class

$$P(C = k | \mathbf{W}, \mathbf{b}, \mathbf{x}) = \frac{e^{\mathbf{w}_k^T \mathbf{x} + b_k}}{\sum_j e^{\mathbf{w}_j^T \mathbf{x} + b_j}}$$

If we consider sample $\mathbf{x}_i$, its class posterior distribution is thus a *categorical* distribution

$$C_i | \mathbf{W}, \mathbf{b}, \mathbf{X}_i = \mathbf{x}_i \sim \text{Cat}(\mathbf{y}_i)$$

where

$$\mathbf{y}_{ik} = \frac{e^{\mathbf{w}_k^T \mathbf{x}_i + b_k}}{\sum_j e^{\mathbf{w}_j^T \mathbf{x}_i + b_j}}$$

Multiclass Logistic Regression

As for the binary case, we can express the log-likelihood for the training class labels as

$$\ell(\mathbf{W}, \mathbf{b}) = \sum_{i=1}^n \log P(C_i = c_i | \mathbf{X}_i = \mathbf{x}_i, \mathbf{W}, \mathbf{b})$$

Remember that the categorical density $P(C_i = c_i | \mathbf{X}_i = \mathbf{x}_i, \mathbf{W}, \mathbf{b})$ can be expressed using a 1-of-K encoding, as

$$\log P(C_i = c_i | \mathbf{X}_i = \mathbf{x}_i, \mathbf{W}, \mathbf{b}) = \log P(C_i = c_i | \mathbf{y}_i) = \sum_{k=1}^K \mathbf{z}_{ik} \log \mathbf{y}_{ik}$$

where $\mathbf{z}_i$ is a vectors that has all component equal to 0, except for the index c_i which is equal to 1

$$\mathbf{z}_i = [0 \dots 0, 1, 0 \dots 0] \text{ , } \mathbf{z}_{ik} = \begin{cases} 1 & \text{if } c_i = k \\ 0 & \text{otherwise} \end{cases}$$

Multiclass Logistic Regression

The log-likelihood can thus be expressed as

$$\ell(\mathbf{W}, \mathbf{b}) = \sum_{i=1}^n \sum_{k=1}^K z_{ik} \log y_{ik}$$

The terms y_{ik} are, again,

$$y_{ik} = \frac{e^{\mathbf{w}_k^T \mathbf{x}_i + b_k}}{\sum_j e^{\mathbf{w}_j^T \mathbf{x}_i + b_j}}$$

and represent the distribution for the class labels according to the Logistic Regression model

Multiclass Logistic Regression

As for the binary case, the expression

$$H(\mathbf{z}_i, \mathbf{y}_i) = - \sum_{k=1}^K z_{ik} \log y_{ik}$$

represents the (multiclass) cross-entropy between the observed and predicted label distributions for sample $\mathbf{x}_i$

As for the binary case, we estimate $\mathbf{W}$ and $\mathbf{b}$ as to maximize the likelihood for the training labels

The ML solution is again the solution that minimizes the (average) cross-entropy:

$$\arg \max_{\mathbf{W}, \mathbf{b}} \ell(\mathbf{W}, \mathbf{b}) = \arg \max_{\mathbf{W}, \mathbf{b}} \left[- \sum_{i=1}^n H(\mathbf{z}_i, \mathbf{y}_i) \right] = \arg \min_{\mathbf{W}, \mathbf{b}} \sum_{i=1}^n H(\mathbf{z}_i, \mathbf{y}_i)$$

Multiclass Logistic Regression

Compared to the binary case, the model is over-parametrized (i.e., we can add a constant vector to all terms w_i without changing the model)

In particular, for a 2-class problem, if we subtract w_2 from both w_1 and w_2 , we recover exactly the binary logistic regression objective.

Multiclass Logistic Regression

Finally, as for the binary class, we can cast the problem as a minimization of a loss function

We rewrite the objective in terms of class labels c as

$$\begin{aligned} J(\mathbf{W}, \mathbf{b}) &= - \sum_{i=1}^n \sum_{k=1}^K z_{ik} \log y_{ik} \\ &= - \sum_{i=1}^n \log \frac{e^{\mathbf{w}_{c_i}^T \mathbf{x}_i + b_{c_i}}}{\sum_{c'=1}^K e^{\mathbf{w}_{c'}^T \mathbf{x}_i + b_{c'}}} \\ &= \sum_{i=1}^n \left[\log \left(\sum_{c'=1}^K e^{\mathbf{w}_{c'}^T \mathbf{x}_i + b_{c'}} \right) - \mathbf{w}_{c_i}^T \mathbf{x}_i - b_{c_i} \right] \\ &= \sum_{i=1}^n l(\mathbf{x}_i, c_i, \mathbf{W}, \mathbf{b}) \end{aligned}$$

l is also called softmax loss

Multiclass Logistic Regression

Again, we can add a regularization term to reduce over-fitting. We thus look for the minimizer of

$$R(\mathbf{W}, \mathbf{b}) = \Omega(\mathbf{W}) + \frac{1}{n}J(\mathbf{W}, \mathbf{b})$$

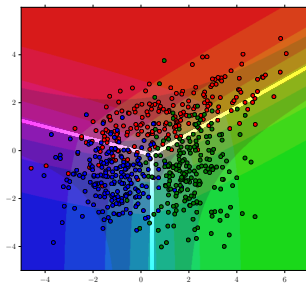
Different regularizers can be used, for example

$$\Omega(\mathbf{w}_1, \dots, \mathbf{w}_N) = \frac{1}{2} \sum_i \|\mathbf{w}_i\|^2$$

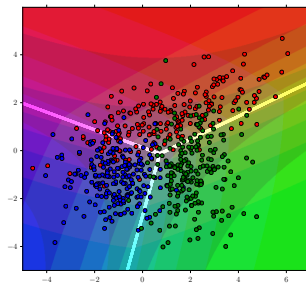
Again, we can replace the average cross-entropy with prior-weighted average cross entropy to account for priors that are different from the empirical training set prior

Multiclass Logistic Regression

Logistic Regression



Gaussian



Multiclass Logistic Regression

MNIST — Error rates for Logistic Regression

DimRed	$\lambda = 0$	$\lambda = 0.00001$	$\lambda = 0.001$	$\lambda = 0.1$	Tied Gau
RAW [768]	8.0%	7.4%	7.9%	12.9%	—
PCA [50]	8.8%	8.8%	8.9%	13.3%	12.6 %
PCA [100]	7.8%	7.8%	8.2%	12.9%	12.3%
PCA+LDA [9]	10.9%	10.9%	11.0%	12.4%	12.3 %

The multiclass logistic regression performs better than the Gaussian model — indeed, the Gaussian assumption is not very accurate for the features we are considering. LogReg only assumes linear separation, but does not impose a distribution over the features

Again, regularization is important, especially when the feature space is large

Multiclass Logistic Regression

Linear logistic regression on MNIST performs better than our Tied-Covariance Gaussian classifier, however it's far worse than our non-linear Gaussian classifier

Remember that, for binary LR, we assumed linear separation surfaces

$$\log \frac{P(C = h_1|\mathbf{x})}{P(C = h_0|\mathbf{x})} = \mathbf{w}^T \mathbf{x} + b$$

which has the same form as the Gaussian classifier with tied covariances

For Gaussian classifier with non-tied covariances we have

$$\log \frac{P(C = h_1|\mathbf{x})}{P(C = h_0|\mathbf{x})} = \mathbf{x}^T \mathbf{A} \mathbf{x} + \mathbf{b}^T \mathbf{x} + c = s(\mathbf{x}, \mathbf{A}, \mathbf{b}, c)$$

Multiclass Logistic Regression

The expression

$$s(\mathbf{x}, \mathbf{A}, \mathbf{b}, c) = \mathbf{x}^T \mathbf{A} \mathbf{x} + \mathbf{b}^T \mathbf{x} + c$$

is quadratic in $\mathbf{x}$, however it is *linear* in $\mathbf{A}$ and $\mathbf{b}$

Indeed, we can rewrite $s(\mathbf{x}, \mathbf{A}, \mathbf{b}, c)$ as

$$s(\mathbf{x}, \mathbf{A}, \mathbf{b}, c) = \langle \mathbf{x} \mathbf{x}^T, \mathbf{A} \rangle + \mathbf{b}^T \mathbf{x} + c$$

where $\langle \cdot, \cdot \rangle$ denotes the inner product

$$\langle \mathbf{A}, \mathbf{B} \rangle = \sum_i \sum_j A_{ij} B_{ij}$$

We can further express $\langle \mathbf{A}, \mathbf{x} \mathbf{x}^T \rangle$ as

$$\langle \mathbf{A}, \mathbf{x} \mathbf{x}^T \rangle = \text{vec}(\mathbf{x} \mathbf{x}^T)^T \text{vec}(\mathbf{A})$$

$\text{vec}(\mathbf{M})$ is the operator that stacks the columns of matrix $\mathbf{M}$

Multiclass Logistic Regression

If we define

$$\phi(\mathbf{x}) = \begin{bmatrix} \text{vec}(\mathbf{x}\mathbf{x}^T) \\ \mathbf{x} \end{bmatrix}$$

and

$$\mathbf{w} = \begin{bmatrix} \text{vec}(\mathbf{A}) \\ \mathbf{b} \end{bmatrix}$$

then the class log-posterior ratio can be expressed as

$$s(\mathbf{x}, \mathbf{w}, c) = \mathbf{w}^T \phi(\mathbf{x}) + c$$

We can thus train a LR model using feature vectors $\phi(\mathbf{x})$ rather than $\mathbf{x}$

We will obtain a model that has linear separation surface in the space defined by the mapping ϕ

This space is also called *expanded feature space*

Multiclass Logistic Regression

The LR model (both binary and multiclass) allows computing linear separation rules for the transformed features $\phi(x)$

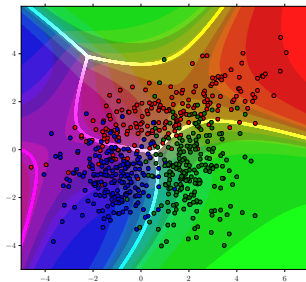
Since expressions $w^T \phi(x) + c$ correspond to quadratic forms in the original feature space, we are actually estimating *quadratic separation surfaces* in the original space

In general, we can consider a transformation $\phi(x)$ of our feature space such that our classes are (approximately) linearly separable in the expanded feature space

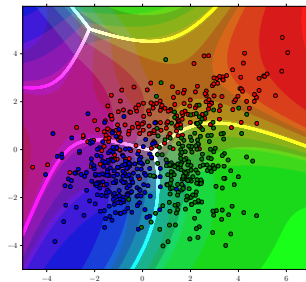
We have to pay attention that the dimensionality of the expanded feature space can grow very quickly — For example, polynomial expansions of degree d result in a feature space of dimensionality $O(M^d)$, where M is the dimensionality of x .

Multiclass Logistic Regression

Logistic Regression



Gaussian



Multiclass Logistic Regression

MNIST — Average pairwise EER for LR with quadratic feature expansion

DimRed	$\lambda = 0$	$\lambda = 1e^{-5}$	$\lambda = 1e^{-3}$	$\lambda = 1e^{-1}$	Gaussian
PCA [50]	1.0%	1.0%	0.9%	1.5%	0.8%

MNIST — Multiclass error rates for LR with quadratic feature expansion

DimRed	$\lambda = 0$	$\lambda = 1e^{-5}$	$\lambda = 1e^{-3}$	$\lambda = 1e^{-1}$	Gaussian
PCA [50]	2.3%	1.9%	1.7%	3.1%	3.6%

In the next lectures we will see a different approach for non-linear classification: Support Vector Machines (SVM)

Alternatively, neural networks (not part of this course) allow jointly estimating a parametric transformation $\phi(\mathbf{x}, \mathbf{\Pi})$ and a classification rule $(\mathbf{w}, b)$ in the transformed space